

3.6 CSS selectors: element, class, ID

"A CSS selector is used to pick an HTML element and give it style."

CSS selector is used style on web page.

There are three fundamental types of CSS selectors.

1. Element Selector
2. Class Selector
3. ID Selector

3.6.0 Element Selector

It selects HTML elements by their name and applies styles to them.

Targets HTML tags directly.

Targets all tags like p, h1, div etc.

Syntax

```
p {  
color: red;  
}
```

3.6.1 Class Selector

Can be used for many elements at once

Selects elements with a specific class

Syntax in CSS: .classname (e.g. .box, .center, .btn)

```
.highlight {  
background: yellow;  
}
```

3.6.2 ID Selector

Each ID must be unique in a page.

affects one specific element

Selects a unique element with a specific id.

Syntax: #idname

```
#title {  
  color: green;  
}
```

3.7 CSS Properties: Color, fonts, text alignment, font-size, Box model: padding, margin, border

Styling Text and Elements

3.7.0 Color

Sets the color of a text

```
P{  
  color: color-name | rgb | rgba | hsl | hsla | hexadecimal;  
}
```

Example (color: red), hexadecimal values (e.g., #FF0000), RGB values (e.g., rgb(255, 0, 0)), or HSL values (e.g., hsl(0, 100%, 50%)).

Sets background color

Sets the background color

```
p {  
  background-color: lightgray;  
}
```

```
p {  
  color: red;  
  background-color: lightgray;  
}
```

Text-align

This property aligns the text in an element at a specific position

text-align: left | right | center | justify | start | end | initial | inherit

```
h1 {  
  text-align: center;  
}
```

3.7.1 Font Size

Sets the size of the text/Font

Setting the text size with pixels (px) gives you full control over the text size.

- em: This unit is relative to the font size of the parent element.
- rem: This unit is relative to the font size of the root HTML element.
- %: This unit is relative to the font size of the parent element
- smaller and larger: These units adjust the font size relative to the parent element.

```
p {  
  font-size: 20px;  
}
```

3.7.2 Box Model

Padding

Space inside the element (between content & border).

It is controlled by the padding property,

You can set padding for each side

padding-top, padding-right, padding-bottom, padding-left

1. padding: 10px; (all sides)
2. padding: 10px 20px; (top-bottom, left-right)
3. padding: 10px 15px 20px 25px; (top, right, bottom, left).

```
div {  
  background-color: blue;  
  padding: 20px;  
}
```

Margin

This is the outermost area, between the element's border and other elements.

They are controlled by the margin property,

You can set margin for each side:

Margin-top, margin-right, margin-bottom, margin-left.

1. `margin: 20px;` (all sides)
2. `margin: 10px 20px;` (top-bottom, left-right)
3. `margin: 10px 15px 20px 25px;` (top, right, bottom, left).

```
div {  
background-color: green;  
margin: 30px;  
}
```

Border that goes around the padding and content

Borders can be applied to each side:

`border-top`, `border-right`, `border-bottom`, `border-left`

Border is always visible and sits between padding and margin.

Three main properties:

- **Width** → thickness (e.g., 10px)
- **Style** → solid, dashed, dotted, double, etc.
 1. **Solid border** → Straight continuous line.
 2. **Dashed border** → Line made of dashes.
 3. **Dotted border** → Line made of dots.
 4. **Double border** → Two parallel lines.
- **Color** → any color (e.g., yellow, blue).
 1. Save files in the same folder:
 2. Save html Code **File Name.html**
 3. `index.html` (File Name :indexl)

```
<!DOCTYPE html>

<html>
<head>

  <title>CSS Border Example </title>

  <link rel="stylesheet" href="New1.css">
</head>
<body>

  <h2>CSS Border </h2>

  <div class="box1">This is a solid border.</div>
  <div class="box2">This is a dashed border.</div>
  .box1 {
    border: 2px solid black;
    padding: 10px;
    margin: 10px;
  }
  .box2 {
    border: 3px dashed red;
    padding: 10px;
    margin: 10px;
  }
  .box3 {
    border: 4px dotted blue;
    padding: 10px;
    margin: 10px;
  }
  .box4 {
    border: 5px double green;
    padding: 10px;
    margin: 10px;
  }
}
```

3.8 Applying styles: inline, internal, external

3.8.0 Inline CSS

Inline CSS has the highest priority compared to Internal and External CSS.

Affects only the specific HTML element where it is applied.

Style is written inside <body> tag

```
<body>

<p style="color: blue; font-size: 16px;">
This is a blue paragraph
</p>
```

3.8.1 Internal CSS

Internal CSS is written inside the <style> tag within the <head> section of an HTML

Internal CSS has less priority than Inline CSS but higher than External CSS.

Styling a single webpage

Style is written inside <style> tag in the <head>.

```
<head>

<title>Internal CSS </title>

<style>

h1 {

    color: red;

    text-align: center;

}

p {

    font-size: 20px;

    color: blue;

}

</style>

</head>
```

3.8.2 External CSS

Style is written in a separate file (style.css).

separate .css file and linked to the HTML document using the <link> element in

<head>

<link rel="stylesheet" href="style.css">

</head>

Best for large projects

Makes code clean, reusable, and easy to maintain.

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>External CSS </title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>This is External CSS</h1>
  <p>This paragraph is styled using External CSS.</p>
</body>
</html>
```

style.css

```
h1 {
  color: purple;
  text-align: center;
}
p {
  font-size: 25px;
  color: red;
}
```

This is External CSS

This paragraph is styled using External CSS.

Assignment 1

Design a web page, which uses padding, margin, and border in External CSS

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS Box Model</title>
  <link rel="stylesheet" href="Style11.css">
</head>
<body>
  <h1>CSS Box Model </h1>
  <div class="box">
    <p>ABES Engg College Ghaziabad</p>
  </div>
</body>
</html>
```

style.css

```
.box {
  width: 400px;
  background-color: yellow;
  padding: 20px;
  border: 25px solid blue;
  margin: 30px;
  text-align: center;
}
```

CSS Box Model



4.0 Display: block, inline, inline-block, float and clear

4.0.0 Display: block

- Block-level elements always start on a new line and take up the full available width of their parent container.
- They respect width, height, margin, and padding properties.
- Examples include div, p, h1, ul, li.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h2>display: block (Headings/Paragraphs)</h2>
    <h1 style="display: block; background: lightblue">
      Amazon - Online Shopping
    </h1>
    <p style="display: block; background: lightgreen">
      This is a full-width paragraph like a blog or product description.
    </p>
  </body>
</html>
```

Output:

display: block (Headings/Paragraphs)

Amazon - Online Shopping

This is a full-width paragraph like a blog or product description.

4.0.1 display: inline

- Inline elements do not start on a new line and only take up as much width as their content requires.
- They do not respect width and height properties. Top and bottom margin and padding are applied but may visually overlap with surrounding content.
- Examples include span, a, strong, em.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h2>display: inline (Navigation links)</h2>
    <a href="#" style="display:inline; margin-right:10px;">Home</a>
    <a href="#" style="display:inline; margin-right:10px;">Deals</a>
    <a href="#" style="display:inline; margin-right:10px;">Cart</a>
  </body>
</html>
```

Output:

display: inline (Navigation links)

[Home](#) [Deals](#) [Cart](#)

4.0.2 display: inline-block

- Inline-block elements combine characteristics of both inline and block elements.
- They do not start on a new line, allowing other elements to sit alongside them.
- They respect width, height, margin, and padding properties, behaving like block elements in this regard.
- This display value is often used for creating grid-like layouts where elements need to sit side-by-side but also require specific dimensions.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body>
    <h2>display: inline-block (Product cards)</h2>
    <div style="display:inline-block; width:150px; height:200px;
background:rgb(182, 182, 47); margin:10px; text-align:center;">
      <h4>Book</h4>
      <p>$10</p>
    </div>
    <div style="display:inline-block;
background:rgb(216, 45, 96); margin:10px; text-align:center;">
      <h4>Headphones</h4>
      <p>$25</p>
    </div>
    <div style="display:inline-block; width:150px; height:200px;
background:lightblue; margin:10px; text-align:center;">
      <h4>Watch</h4>
      <p>$50</p>
    </div>
  </html>
```

Output:

display: inline-block (Product cards)



4.0.3 float

- The float property takes an element out of the normal document flow and positions it to the left or right within its containing element.
- Content (like text) will wrap around the floated element.
- Commonly used for image and text wrapping or creating multi-column layouts, though modern CSS layout methods like Flexbox and Grid are often preferred for complex layouts.
- Values include left, right, none.

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>
  <body bgcolor="pink">
    <h2>float (Image in an article)</h2>
    
    <p>
This text flows around the image, just like a newspaper <br>or blog post with an
image aligned to the left.
It continues <br>to wrap neatly around the floated image.
    </p>
  </body>
</html>
```

Output:

float (Image in an article)



This text flows around the image, just like a newspaper or blog post with an image aligned to the left. It continues to wrap neatly around the floated image.

4.0.4 clear

- The clear property is used in conjunction with float to control the positioning of elements that follow a floated element.
- It prevents an element from sitting alongside a floated element on the specified side.
- Values include left (clears floats on the left), right (clears floats on the right), both (clears floats on both sides), and none (default, allows floats on both sides).
- Often used to prevent content from wrapping around a floated element or to ensure a container expands to encompass its floated children (e.g., using a "clearfix" hack).

Example:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Document</title>
  </head>
  <body bgcolor="pink">
    <h2>clear (Footer below floated content)</h2>
    
    <p>This paragraph wraps around the floated image.</p>
    <p style="clear:both; background:lightgray;">This is the footer section,
forced below the floated image using clear:both.</p>
  </body>
</html>
```

Output:

clear (Footer below floated content)



This paragraph wraps around the floated image.

This is the footer section, forced below the floated image using clear:both.

4.1 Introduction to Flexbox: justify-content, align-items, flex-direction

Flexbox

Flexbox (Flexible Box Layout) is a modern CSS3 layout system designed to align, distribute, and organize elements in a container efficiently. It is extremely useful for creating responsive web applications and solving alignment issues that older techniques (like floats and inline-block) could not handle.

4.1.0 Introduction to Flexbox

Flexbox works on a parent-child relationship where the parent (container) uses flex properties, and the children (items) adjust themselves accordingly. Flexbox provides control in two axes:

- **Main Axis** – controlled by flex-direction and justify-content.
- **Cross Axis** – controlled by align-items.

Example:

```
.container {  
  display: flex; /* flexbox activate */  
}
```

4.1.1 flex-direction

The 'flex-direction' property defines the direction of the main axis of the flex container.

Values:

- **row (default)**: Items placed left to right.
- **row-reverse**: Items placed right to left.
- **column**: Items placed top to bottom.
- **column-reverse**: Items placed bottom to top.

Example:

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

4.1.2 justify-content

The 'justify-content' property aligns flex items along the main axis (horizontal for row, vertical for column).

Values:

- **flex-start:** Items **aligned at the start**.
- **flex-end:** Items aligned at the end.
- **center:** Items aligned at the center.
- **space-between:** Equal space between items; first and last at edges.
- **space-around:** Equal space around items with half space at edges.
- **space-evenly:** Equal space between all items including edges.

Example:

```
.container {  
  display: flex;  
  justify-content: space-between;  
}
```

4.1.3 align-items

The 'align-items' property aligns items along the cross axis (perpendicular to main axis).

Values:

- **stretch (default):** Items stretch to fill container.
- **flex-start:** Items align to start of cross axis.
- **flex-end:** Items align to end of cross axis.
- **center:** Items aligned at center of cross axis.
- **baseline:** Items align based on text baseline.

Example:

```
.container {  
  display: flex;  
  justify-content: space-between;  
}
```

4.1.4 Example

Below is an example showing flex-direction, justify-content, and align-items working together:

Example:

Index.html

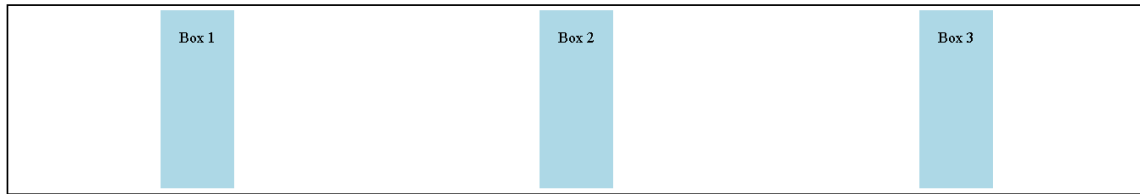
```
<!DOCTYPE html>
<html>
<head>
  <title>Flexbox Example</title>
  <!-- CSS file ko link karna -->
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
  <div class="container">
    <div class="item">Box 1</div>
    <div class="item">Box 2</div>
    <div class="item">Box 3</div>
  </div>
</body>
</html>
```

Stylesheet.css

```
.container {
  display: flex;          /* Flexbox activate */
  flex-direction: row;    /* Items left → right */
  justify-content: space-around; /* Horizontal equal spacing */
  align-items: center;    /* Vertical center alignment */
  height: 200px;         /* Container height */
  border: 2px solid black; /* Border around container */
}

.item {
  background: lightblue; /* Box background color */
  padding: 20px;        /* Inner spacing */
  margin: 5px;          /* Outer spacing between boxes */
}
```

Output



4.1.5 Advantages of Flexbox

- Simplifies centering both horizontally and vertically.
- Provides responsive layouts without using floats or positioning hacks.
- Allows reordering of items easily.
- Adjusts automatically to different screen sizes.

4.1.6 Real-life Use Cases

- Navigation bars using justify-content: space-between.
- Login page content aligned using align-items: center.
- Card layouts with row direction and wrapping.
- Footers distributed evenly using space-evenly.

4.2 Units: px, %, em, rem, vh, vw, Media queries and breakpoints

4.2.0 Pixel (px)

Definition

- px stands for **pixels**.
- It is an **absolute unit** (fixed size).
- 1px = 1 dot on the screen (depending on screen resolution).

Example

```
p {  
  font-size: 16px;  
}
```